

# 中国科学院大学

## 《计算机网络(研讨课)》实验报告

姓名 寇逸欣 学号 2023K8009922004 专业 计算机科学与技术  
实验项目编号 07 实验名称 路由器转发实验

### 一、实验内容

1. 基于现有框架,实现路由器转发机制。
2. 在 PPT 的星型拓扑 (h1-r1,h2-r1,h3-r1) 中使用 ping 验证并返回正确内容,3 个 ping 通 +host unreachable+net unreachable,共 5 个测试
3. 自己编写多跳拓扑并自行配置,见 PPT(h1-r1-r2-h2),在其中使用 ping 和 traceroute 验证,共 2 个测试。

### 二、实验流程

#### 2.1 处理 ip 数据包 (包括转发)

如果接收到的数据包是 ICMP echo 请求,而且目标地址和当前接口的 IP 地址相同,则发送 ICMP echo 回复;否则,进行数据包转发,把数据包发送到下一跳。在进行转发的时候,需要进行最长前缀匹配,找到下一跳的路由条目。如果 gw 是 0 则说明目的地址在同一网段,下一跳地址即为目的地址,否则下一跳地址为 gw。如果找不到匹配的路由条目,则发送 ICMP 目的地不可达消息;如果 TTL 值减 1 后为 0,则发送 ICMP 时间超过消息;否则,更新 IP 头的校验和,并通过 ARP 协议将数据包发送到下一跳。同时,把 RTT 值减 1,并重新计算校验和,如果 RTT 值为 0,则发送 ICMP 时间超过消息。

注意,解析 dest\_ip 后要使用 ntohs 函数将网络字节序转换为主机字节序,以确保正确的比较和处理。

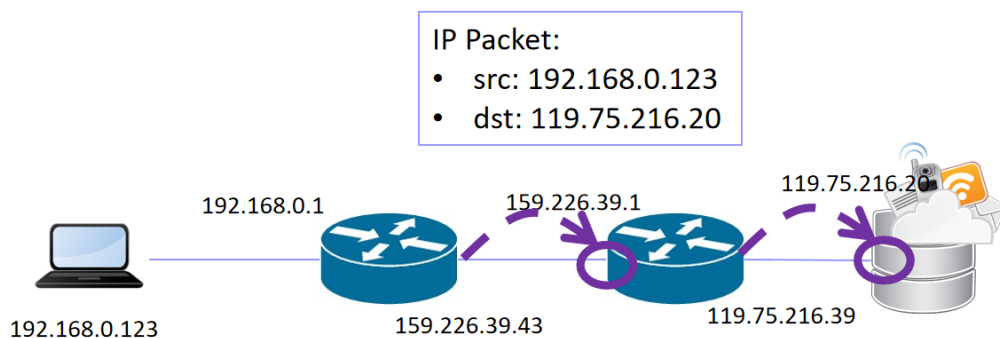


图 1: 传递过程示例

```
void handle_ip_packet(iface_info_t *iface, char *packet, int len)
{
    // fprintf(stderr, "TODO: handle ip packet.\n");
    struct iphdr *ip_hdr = packet_to_ip_hdr(packet);
    u32 dest_ip = ntohs(ip_hdr->daddr);
    u32 iface_ip = iface->ip;
    if (dest_ip == iface_ip) {
        if (ip_hdr->protocol == IPPROTO_ICMP) {
```

```

    struct icmphdr *icmp_hdr = (struct icmphdr *)IP_DATA(ip_hdr);
    if (icmp_hdr->type == ICMP_ECHOREQUEST) {
        icmp_send_packet(packet, len, ICMP_ECHOREPLY, 0);
    }
}
} else {
    // forward the packet
    rt_entry_t *entry = longest_prefix_match(dest_ip);
    if (entry == NULL) {
        icmp_send_packet(packet, len, ICMP_DEST_UNREACH, ICMP_NET_UNREACH);
    } else {
        if (--ip_hdr->ttl == 0) {
            icmp_send_packet(packet, len, ICMP_TIME_EXCEEDED, ICMP_EXC_TTL);
        } else {
            u32 nxt_hop = entry->gw == 0 ? dest_ip : entry->gw;
            ip_hdr->checksum = ip_checksum(ip_hdr);
            iface_send_packet_by_arp(entry->iface, nxt_hop, packet, len);
        }
    }
}
}
}

```

## 2.2 IP 前缀查找和发送 IP 数据包

对于最长前缀匹配,  $(dst\_ip \& mask) == (dest \& mask)$ , 且掩码长度最长 (mask 值最大) 的路由条目即为匹配结果。

```

rt_entry_t *longest_prefix_match(u32 dst)
{
    rt_entry_t *entry = NULL;
    rt_entry_t *best_entry = NULL;
    u32 max_mask = 0;
    list_for_each_entry(entry, &rtable, list) {
        if ((dst & entry->mask) == (entry->dest & entry->mask)) {
            if (entry->mask > max_mask) {
                max_mask = entry->mask;
                best_entry = entry;
            }
        }
    }
    return best_entry;
}

```

对于 IP 数据包的发送, 首先要进行最长前缀匹配寻找下一跳的路由条目, 如果 gw 不为 0, 则下一跳地址为 gw, 否则为目的地址 dest\_ip。

```

void ip_send_packet(char *packet, int len)
{
    struct iphdr *ip_hdr = packet_to_ip_hdr(packet);

```

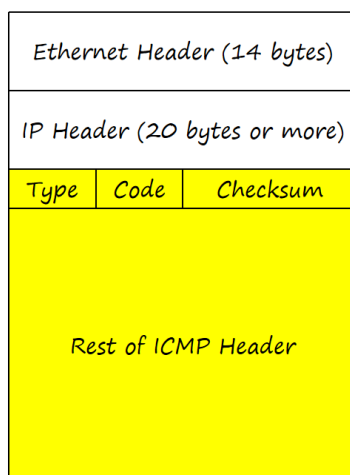
```

u32 destip = ntohl(ip_hdr->daddr);
rt_entry_t *entry = longest_prefix_match(destip);
if (entry->gw) {
    iface_send_packet_by_arp(entry->iface, entry->gw, packet, len);
} else {
    iface_send_packet_by_arp(entry->iface, destip, packet, len);
}
}

```

## 2.3 发送 ICMP 数据包

首先,解析输入的 IP 数据包,获取目的 IP 和源 IP,切换为主机字节序。然后根据 ICMP 的类型确定数据包的长度。如果是 ICMP 应答,则长度和输入的数据包相同,如果是目的不可达或者超时,则需要构造一个新的数据包,长度为以太网头 +IP 头 +ICMP 头 + 原始 IP 头 +8 字节数据。接着,分配内存用于发送数据包,并初始化以太网头和 IP 头。以太网头的类型字段设置为 IP 协议类型。然后,初始化 IP 头,源地址为当前接口的 IP 地址,目的地址为输入数据包的源地址,长度为数据包总长度减去以太网头长度,协议字段设置为 ICMP。接下来,设置 ICMP 头的类型和代码字段。如果是 ICMP 应答,则将输入数据包的 ICMP 数据部分复制到发送数据包中;如果是目的不可达或者超时,则将输入数据包的 IP 头和前 8 字节数据复制到发送数据包中,并在 ICMP 数据部分填充 4 个字节的 0。最后,计算 ICMP 头的校验和,并调用 ip\_send\_packet 函数发送数据包。



### 路由表查找失败

Type: 3, Code: 0, Rest of ICMP Header: 前4字节设置为0, 接着拷贝收到数据包的IP头部 (≥ 20字节) 和随后的8字节

### ARP查询失败

Type: 3, Code: 1, Rest of ICMP Header: 同上

### TTL值减为0

Type: 11, Code: 0, Rest of ICMP Header: 同上

### 收到Ping本端口的数据包 (Type为8)

Type: 0, Code: 0, Rest of ICMP Header: 拷贝Ping包中的相应字段

图 2: ICMP 协议格式

```

void icmp_send_packet(const char *in_pkt, int len, u8 type, u8 code)
{
    struct iphdr* in_ip_hdr = packet_to_ip_hdr(in_pkt);
    int packet_len;
    if(type == ICMP_ECHOREPLY){
        packet_len = len;
    }else{
        packet_len = ETHER_HDR_SIZE + IP_BASE_HDR_SIZE + ICMP_HDR_SIZE + IP_HDR_SIZE(in_ip_hdr) + 8;
    }
}

```

```

char *send_pkt = malloc(packet_len * sizeof(char));
struct ether_header *eh = (struct ether_header *) send_pkt;
struct iphdr *iph = packet_to_ip_hdr(send_pkt);
struct icmphdr *icmph = (struct icmphdr *) (send_pkt + ETHER_HDR_SIZE + IP_BASE_HDR_SIZE);
eh->ether_type = htons(ETH_P_IP);

rt_entry_t *entry = longest_prefix_match(ntohl(in_ip_hdr->saddr));
ip_init_hdr(iph, entry->iface->ip, ntohl(in_ip_hdr->saddr), packet_len - ETHER_HDR_SIZE, 1);

icmph->code = code;
icmph->type = type;

if(type == 0){
    memcpy(send_pkt + ETHER_HDR_SIZE + IP_HDR_SIZE(iph) + 4, in_pkt + ETHER_HDR_SIZE +
        IP_HDR_SIZE(in_ip_hdr) + 4, packet_len - (ETHER_HDR_SIZE + IP_HDR_SIZE(in_ip_hdr) + 4));
}else{
    memset(send_pkt + ETHER_HDR_SIZE + IP_HDR_SIZE(iph) + 4, 0, 4);
    memcpy(send_pkt + ETHER_HDR_SIZE + IP_HDR_SIZE(iph) + 4 + 4, in_ip_hdr, IP_HDR_SIZE(in_ip_hdr)
        + 8);
}
icmph->checksum = icmp_checksum(icmph, packet_len - ETHER_HDR_SIZE - IP_BASE_HDR_SIZE);

ip_send_packet(send_pkt, packet_len);
}

```

## 2.4 ARP 缓存相关操作

ARP 缓存操作有查询, 暂存, 插入和老化四个部分。

对于查询, 遍历 ARP 缓存链表, 查找与输入 IP 地址匹配的 ARP 缓存项, 如果找到则将对应的 MAC 地址复制到输出参数中, 并返回 1 表示成功; 如果未找到则返回 0 表示失败。

```

int arpcache_lookup(u32 ip4, u8 mac[ETH_ALEN])
{
    pthread_mutex_lock(&arpcache.lock);
    for (int i = 0; i < MAX_ARP_SIZE; i++) {
        if (arpcache.entries[i].valid && arpcache.entries[i].ip4 == ip4) {
            memcpy(mac, arpcache.entries[i].mac, ETH_ALEN);
            pthread_mutex_unlock(&arpcache.lock);
            return 1;
        }
    }

    pthread_mutex_unlock(&arpcache.lock);
    return 0;
}

```

对于暂存, 首先, 检查是否已经存在相同 IP 地址的 ARP 请求项, 如果存在则将新的数据包添加到该请求项的等待队列中, 并返回; 如果不存在, 则创建一个新的 ARP 请求项, 将数据包添加到等待队列中, 并将该请求项添加到 ARP 请求链表中。

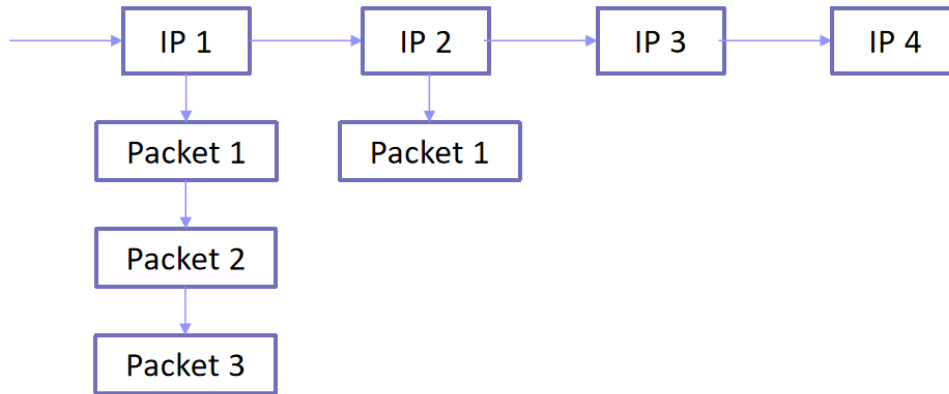


图 3: 请求链表

```

void arpcache_append_packet(iface_info_t *iface, u32 ip4, char *packet, int len)
{
    pthread_mutex_lock(&arpcache.lock);
    struct arp_req *req_entry = NULL;
    list_for_each_entry(req_entry, &arpcache.req_list, list) {
        if (req_entry->ip4 == ip4 && req_entry->iface == iface) {
            struct cached_pkt *new_pkt = (struct cached_pkt *)malloc(sizeof(struct cached_pkt));
            new_pkt->packet = (char *)malloc(len);
            memcpy(new_pkt->packet, packet, len);
            new_pkt->len = len;
            list_add_tail(&new_pkt->list, &req_entry->cached_packets);
            pthread_mutex_unlock(&arpcache.lock);
            return;
        }
    }

    // not found, create a new arp_req entry
    req_entry = (struct arp_req *)malloc(sizeof(struct arp_req));
    req_entry->iface = iface;
    req_entry->ip4 = ip4;
    req_entry->sent = time(NULL);
    req_entry->retries = 0;
    init_list_head(&req_entry->cached_packets);
    list_add_tail(&req_entry->list, &arpcache.req_list);

    struct cached_pkt *new_pkt = (struct cached_pkt *)malloc(sizeof(struct cached_pkt));
    new_pkt->packet = (char *)malloc(len);
    memcpy(new_pkt->packet, packet, len);
    new_pkt->len = len;
    list_add_tail(&new_pkt->list, &req_entry->cached_packets);

    pthread_mutex_unlock(&arpcache.lock);
    arp_send_request(iface, ip4);
}

```

对于插入, 查找一个空闲的缓存项, 将新的 IP 地址和 MAC 地址存入该缓存项, 并设置其为有效状态; 如果没有空闲的缓存项, 则随机选择一个缓存项进行替换。然后发送所有等待该 IP 地址的 ARP 请求项中的数据包, 并删除对应的 ARP 请求条目。

```
void arpcache_insert(u32 ip4, u8 mac[ETH_ALEN])
{
    pthread_mutex_lock(&arpcache.lock);

    // insert arp cache
    int inserted = 0;
    for (int i = 0; i < MAX_ARP_SIZE; i++) {
        if (!arpcache.entries[i].valid) {
            arpcache.entries[i].ip4 = ip4;
            memcpy(arpcache.entries[i].mac, mac, ETH_ALEN);
            arpcache.entries[i].added = time(NULL);
            arpcache.entries[i].valid = 1;
            inserted = 1;
            break;
        }
    }
    if (!inserted) {
        srand(time(NULL));
        int idx = rand() % MAX_ARP_SIZE;
        arpcache.entries[idx].ip4 = ip4;
        memcpy(arpcache.entries[idx].mac, mac, ETH_ALEN);
        arpcache.entries[idx].added = time(NULL);
        arpcache.entries[idx].valid = 1;
    }

    // send pending packets
    struct arp_req *req_entry = NULL, *req_q;
    list_for_each_entry_safe(req_entry, req_q, &arpcache.req_list, list) {
        if (req_entry->ip4 == ip4) {
            struct cached_pkt *pkt_entry = NULL, *pkt_q;
            list_for_each_entry_safe(pkt_entry, pkt_q, &req_entry->cached_packets, list) {
                struct ether_header *eh = (struct ether_header *)pkt_entry->packet;
                memcpy(eh->ether_dhost, mac, ETH_ALEN);
                iface_send_packet(req_entry->iface, pkt_entry->packet, pkt_entry->len);
                list_delete_entry(&pkt_entry->list);
                // free(pkt_entry->packet);
                free(pkt_entry);
            }
            list_delete_entry(&req_entry->list);
            free(req_entry);
        }
    }

    pthread_mutex_unlock(&arpcache.lock);
}
```

最后还要实现一个 `arpcache_sweep` 函数，用于老化 ARP 缓存项和重发 ARP 请求。该函数每秒钟执行一次，遍历 ARP 缓存项，如果存在条目超过设定时间则标记为无效。同时，遍历请求链表，如果请求项发送时间超过 1 秒且重试次数小于 5 次，则重新发送 ARP 请求并更新发送时间和重试次数；如果重试次数达到 5 次，则发送 ICMP 目的地不可达消息，并删除该请求项。

```
void *arpcache_sweep(void *arg)
{
    while (1) {
        sleep(1);
        pthread_mutex_lock(&arpcache.lock);
        time_t now = time(NULL);

        // sweep arp cache entries
        for (int i = 0; i < MAX_ARP_SIZE; i++) {
            if (arpcache.entries[i].valid && now - arpcache.entries[i].added > ARP_ENTRY_TIMEOUT) {
                arpcache.entries[i].valid = 0;
            }
        }

        // sweep arp requests
        struct arp_req *req_entry = NULL, *req_q;
        list_for_each_entry_safe(req_entry, req_q, &arpcache.req_list, list) {
            if (now - req_entry->sent >= 1) {
                if (req_entry->retries >= ARP_REQUEST_MAX_RETRIES) {
                    // send icmp host unreachable for each pending packet
                    struct cached_pkt *pkt_entry = NULL, *pkt_q;
                    list_for_each_entry_safe(pkt_entry, pkt_q, &req_entry->cached_packets, list) {
                        pthread_mutex_unlock(&(arpcache.lock));
                        icmp_send_packet(pkt_entry->packet, pkt_entry->len, ICMP_DEST_UNREACH,
                                         ICMP_HOST_UNREACH);
                        pthread_mutex_lock(&(arpcache.lock));
                        list_delete_entry(&pkt_entry->list);
                        free(pkt_entry->packet);
                        free(pkt_entry);
                    }
                    // delete this arp_req entry
                    list_delete_entry(&req_entry->list);
                    free(req_entry);
                } else {
                    // resend arp request
                    arp_send_request(req_entry->iface, req_entry->ip4);
                    req_entry->sent = now;
                    req_entry->retries += 1;
                }
            }
        }

        pthread_mutex_unlock(&arpcache.lock);
    }
    return NULL;
}
```

```
}

```

注意，以上几个函数都需要加锁以确保线程安全。这就引出了一个问题，即在 `arp_cache_sweep` 函数中调用 `icmp_send_packet` 函数时，可能会导致死锁。为了解决这个问题，我们在调用 `icmp_send_packet` 函数之前释放锁，调用之后重新获取锁。（这点在后续的 `bugfix` 中有体现）

## 2.5 发送 ARP 请求和应答

对于 ARP 的请求和应答，我们需要按照 ARP 协议的格式，分别构造 ARP 请求包和 ARP 应答包，并调用 `iface_send_packet` 函数发送出去。

|                         |                   |                    |
|-------------------------|-------------------|--------------------|
| Dest Ether Addr         |                   |                    |
| Dest Ether Addr (cont.) | Src Ether Addr    |                    |
| Src Ether Addr (cont.)  |                   |                    |
| Proto Type (0x0806)     | ARP Header (0x01) |                    |
| ARP Proto (0x0800)      | HW Addr Len (6)   | Proto Addr Len (4) |
| ARP Operation Type      | Sender HW Addr    |                    |
| Sender HW Addr (cont.)  |                   |                    |
| Sender Proto Addr       |                   |                    |
| Target HW Addr          |                   |                    |
| Target HW Addr (cont.)  | Target Proto Addr |                    |
| Target Proto Addr       |                   |                    |

图 4: ARP 请求和应答格式

```
void arp_send_request(iface_info_t *iface, u32 dst_ip)
{
    char *packet = malloc(sizeof(struct ether_header) + sizeof(struct ether_arp));
    struct ether_header *eh = (struct ether_header *)packet;
    struct ether_arp *arp_hdr = (struct ether_arp *) (packet + sizeof(struct ether_header));
    // fill ethernet header
    memset(eh->ether_dhost, 0xff, ETH_ALEN); // dest mac: broadcast
    memcpy(eh->ether_shost, iface->mac, ETH_ALEN); // src mac
    eh->ether_type = htons(ETH_P_ARP); // type: 0806 ARP
    // fill arp header
    arp_hdr->arp_hrd = htons(ARPHRD_ETHER); // hardware type: Ethernet
    arp_hdr->arp_pro = htons(ETH_P_IP); // protocol type: 0800 IPV4
    arp_hdr->arp_hln = ETH_ALEN; // hardware address length: 6
    arp_hdr->arp_pln = 4; // protocol address length: 4
    arp_hdr->arp_op = htons(ARPOP_REQUEST); // operation: 1 request
    memcpy(arp_hdr->arp_sha, iface->mac, ETH_ALEN); // sender mac
    arp_hdr->arp_spa = htonl(iface->ip); // sender ip
    memset(arp_hdr->arp_tha, 0x00, ETH_ALEN); // target mac: unknown
    arp_hdr->arp_tpa = htonl(dst_ip); // target ip
    iface_send_packet(iface, packet, sizeof(struct ether_header) + sizeof(struct ether_arp)); //
    send packet
}
```

```
}
```

应答包的构造类似,这里不再赘述。

随后我们需要实现一个处理 ARP 包的函数 `handle_arp_packet`, 用于处理接收到的 ARP 请求和应答包。如果是 ARP 请求包, 且目标 IP 地址与当前接口的 IP 地址相同, 则将发送方的 IP 地址和 MAC 地址插入到 ARP 缓存中并发送 ARP 应答包; 如果是 ARP 应答包, 则只将发送方的 IP 地址和 MAC 地址插入到 ARP 缓存中。

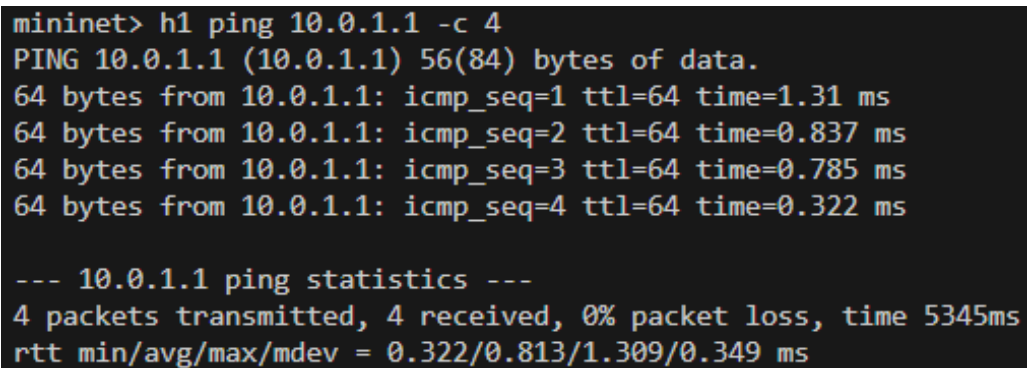
```
void handle_arp_packet(iface_info_t *iface, char *packet, int len)
{
    if (len < sizeof(struct ether_header) + sizeof(struct ether_arp)) {
        return;
    }

    struct ether_arp *arp_hdr = (struct ether_arp *) (packet + sizeof(struct ether_header));
    u16 op = ntohs(arp_hdr->arp_op);
    u32 spa = ntohl(arp_hdr->arp_spa);
    u32 tpa = ntohl(arp_hdr->arp_tpa);

    if (op == ARPOP_REQUEST) {
        if (tpa == iface->ip) {
            arpcache_insert(spa, arp_hdr->arp_sha);
            arp_send_reply(iface, arp_hdr);
        }
    }
    else if (op == ARPOP_REPLY) {
        arpcache_insert(spa, arp_hdr->arp_sha);
    }
}
```

## 三、实验结果

### 3.1 实验内容一: 在 h1 上进行 ping 实验



```
mininet> h1 ping 10.0.1.1 -c 4
PING 10.0.1.1 (10.0.1.1) 56(84) bytes of data.
64 bytes from 10.0.1.1: icmp_seq=1 ttl=64 time=1.31 ms
64 bytes from 10.0.1.1: icmp_seq=2 ttl=64 time=0.837 ms
64 bytes from 10.0.1.1: icmp_seq=3 ttl=64 time=0.785 ms
64 bytes from 10.0.1.1: icmp_seq=4 ttl=64 time=0.322 ms

--- 10.0.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 5345ms
rtt min/avg/max/mdev = 0.322/0.813/1.309/0.349 ms
```

图 5: h1 ping r1 结果

Ping 10.0.1.1 (r1), 能够 ping 通。

```

mininet> h1 ping 10.0.2.22 -c 4
PING 10.0.2.22 (10.0.2.22) 56(84) bytes of data.
64 bytes from 10.0.2.22: icmp_seq=1 ttl=63 time=0.412 ms
64 bytes from 10.0.2.22: icmp_seq=2 ttl=63 time=0.157 ms
64 bytes from 10.0.2.22: icmp_seq=3 ttl=63 time=0.738 ms
64 bytes from 10.0.2.22: icmp_seq=4 ttl=63 time=0.415 ms

--- 10.0.2.22 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3423ms
rtt min/avg/max/mdev = 0.157/0.430/0.738/0.206 ms

```

图 6: h1 ping h2 结果

Ping 10.0.2.22 (h2), 能够 ping 通。

```

mininet> h1 ping 10.0.3.33 -c 4
PING 10.0.3.33 (10.0.3.33) 56(84) bytes of data.
64 bytes from 10.0.3.33: icmp_seq=1 ttl=63 time=0.651 ms
64 bytes from 10.0.3.33: icmp_seq=2 ttl=63 time=0.208 ms
64 bytes from 10.0.3.33: icmp_seq=3 ttl=63 time=0.395 ms
64 bytes from 10.0.3.33: icmp_seq=4 ttl=63 time=0.289 ms

--- 10.0.3.33 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3219ms
rtt min/avg/max/mdev = 0.208/0.385/0.651/0.166 ms

```

图 7: h1 ping h3 结果

Ping 10.0.3.33 (h3), 能够 ping 通。

```

mininet> h1 ping 10.0.3.11 -c 4
PING 10.0.3.11 (10.0.3.11) 56(84) bytes of data.
From 10.0.1.1 icmp_seq=1 Destination Host Unreachable
From 10.0.1.1 icmp_seq=2 Destination Host Unreachable
From 10.0.1.1 icmp_seq=3 Destination Host Unreachable
From 10.0.1.1 icmp_seq=4 Destination Host Unreachable

--- 10.0.3.11 ping statistics ---
4 packets transmitted, 0 received, +4 errors, 100% packet loss, time 3163ms
pipe 4

```

图 8: h1 ping unreachable host 结果

Ping 10.0.3.11, 返回 ICMP Destination Host Unreachable。

```

mininet> h1 ping 10.0.4.1 -c 4
PING 10.0.4.1 (10.0.4.1) 56(84) bytes of data.
From 10.0.1.1 icmp_seq=1 Destination Net Unreachable
From 10.0.1.1 icmp_seq=2 Destination Net Unreachable
From 10.0.1.1 icmp_seq=3 Destination Net Unreachable
From 10.0.1.1 icmp_seq=4 Destination Net Unreachable

--- 10.0.4.1 ping statistics ---
4 packets transmitted, 0 received, +4 errors, 100% packet loss, time 3436ms

```

图 9: h1 ping unreachable net 结果

Ping 10.0.4.1, 返回 ICMP Destination Net Unreachable。

### 3.2 实验内容二:构造一个包含多个路由器节点组成的网络,进行连通性测试和路径测试

构造了一个包含两个主机节点和三个路由节点的网络拓扑结构,并且手动配置了路由表。

```
# Get nodes
h1, h2, r1, r2, r3 = net.get('h1', 'h2', 'r1', 'r2', 'r3')

# Configure hosts
h1.cmd('ifconfig h1-eth0 10.0.1.11/24')
h2.cmd('ifconfig h2-eth0 10.0.4.22/24')
h1.cmd('route add default gw 10.0.1.1')
h2.cmd('route add default gw 10.0.4.1')

# Configure routers
r1.cmd('ifconfig r1-eth0 10.0.1.1/24')
r1.cmd('ifconfig r1-eth1 10.0.2.1/24')
r2.cmd('ifconfig r2-eth0 10.0.2.2/24')
r2.cmd('ifconfig r2-eth1 10.0.3.1/24')
r3.cmd('ifconfig r3-eth0 10.0.3.2/24')
r3.cmd('ifconfig r3-eth1 10.0.4.1/24')

# Configure static routes on routers
r1.cmd('route add -net 10.0.3.0/24 gw 10.0.2.2')
r1.cmd('route add -net 10.0.4.0/24 gw 10.0.2.2')

r2.cmd('route add -net 10.0.1.0/24 gw 10.0.2.1')
r2.cmd('route add -net 10.0.4.0/24 gw 10.0.3.2')

r3.cmd('route add -net 10.0.1.0/24 gw 10.0.3.1')
r3.cmd('route add -net 10.0.2.0/24 gw 10.0.3.1')

# Disable offloading and IPv6
for n in (h1, h2, r1, r2, r3):
    n.cmd('./scripts/disable_offloading.sh')
    n.cmd('./scripts/disable_ipv6.sh')
```

连通性测试结果如下 (以 h1 ping r1 为例):

```
mininet> h1 ping 10.0.2.1 -c 3
PING 10.0.2.1 (10.0.2.1) 56(84) bytes of data.
64 bytes from 10.0.2.1: icmp_seq=1 ttl=64 time=4.38 ms
64 bytes from 10.0.2.1: icmp_seq=2 ttl=64 time=0.031 ms
64 bytes from 10.0.2.1: icmp_seq=3 ttl=64 time=0.083 ms

--- 10.0.2.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2111ms
rtt min/avg/max/mdev = 0.031/1.499/4.384/2.039 ms
```

图 10: h1 ping r1 结果

路径测试结果如下 (h1 traceroute h2):

```
mininet> h1 traceroute 10.0.4.22
traceroute to 10.0.4.22 (10.0.4.22), 30 hops max, 60 byte packets
 1  10.0.1.1 (10.0.1.1)  4.148 ms  4.124 ms  4.116 ms
 2  10.0.2.2 (10.0.2.2)  4.423 ms  5.939 ms  6.216 ms
 3  10.0.3.2 (10.0.3.2)  7.803 ms  12.984 ms  13.020 ms
 4  10.0.4.22 (10.0.4.22)  13.017 ms  13.014 ms  13.009 ms
```

图 11: h1 traceroute h2 结果

## 四、 实验中遇到的问题

本次实验中主要遇到了两个 bug:

### 4.1 死锁问题

和前文所述一致,在 `arpcache_sweep` 函数中调用 `icmp_send_packet` 函数前需要先释放锁,否则可能会导致死锁。因为, `arpcache_sweep` 函数持有 `arpcache` 锁,而 `icmp_send_packet` 函数最终会调用 `iface_send_packet_by_arp` 函数,而 `iface_send_packet_by_arp` 函数中会调用 `arpcache_lookup` 函数,从而尝试获取已经被持有的锁,导致死锁。

问题的发现也是在回传 ICMP 应答包的时候,发现始终没有相应。然后通过插入 log 输出,发现程序虽然成功进入了 `icmp_send_packet` 函数,但是并没有成功返回。经过分析代码逻辑,最终发现了这个死锁问题。

### 4.2 ICMP 数据包构造问题

在发送 ICMP 目的地不可达和时间超过消息时,ICMP 数据包的构造有误,导致接收端无法正确解析 ICMP 消息。具体来说,在构造 ICMP 数据包时,由于需要把源 IP 地址设置为当前接口的 IP 地址,目的 IP 地址设置为输入数据包的源 IP 地址,但是在之前的实现中,错误地将源 IP 地址和目的 IP 地址搞反了,导致接收端无法正确识别 ICMP 消息。

问题的发现是在 ICMP 回传时,发现虽然包正确的发送出去了,但是接收端并没有收到正确的 ICMP 消息。通过将 packet 写入文件中,我查看了发送的包数据,发现了这个问题。